浅谈 SKILL 研发的最佳实践——以百补详情助手为例

原创 天猫技术团队 大淘宝技术 2026年8月20日 19:47 浙江 33人

本文以百亿补贴详情助手 Skill 的迭代历程为例，总结 Skill 研发与迭代实践。文章先梳理业界方法论——Claude Skills 的渐进式披露与控制调优、MiniMax 的工业化规范、以及常见的 Skill 设计模式；随后从**架构设计、研发效能、运行优化** 3 个维度展开详细介绍了桥接器解耦本地 Skill 与远程 Agent、按需共享上下文、优先输出优质 HTML 报告等高效实践，帮助读者提高 Skill 研发效能。

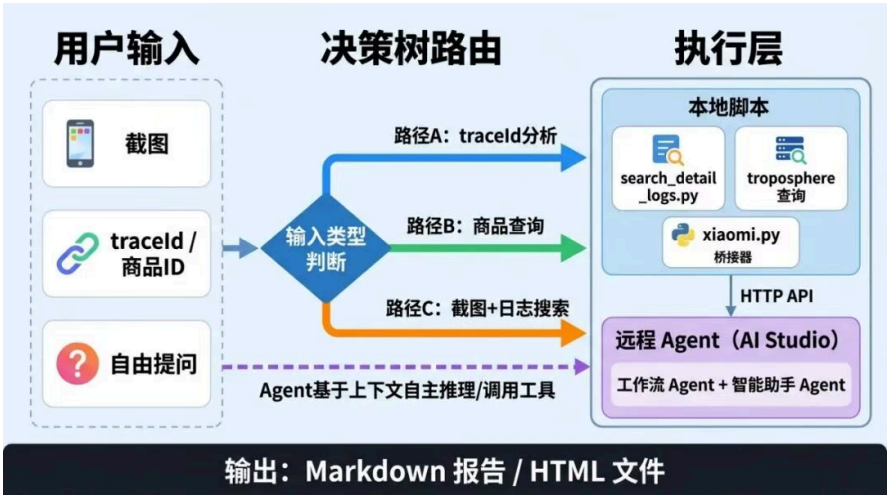
01 背景

自从 Skill 概念被 Anthropic 提出，我所在的团队已开发 50+ Skill，涵盖补贴查询/创建/分析、舆情分析、频道巡检、智能圈品、素材生成、代码助手、商品诊断等场景。这一路做下来，我们也读了大量讲 Skill 最佳实践的文章和仓库，比如 Claude Skills 官方文档[1]、MiniMax 开源仓库[2]、Agent Skills 开放标准[3]，慢慢发现一个共同点：**对怎么写好 `SKILL.md` 的标准讲得多，但对 Skill 如何研发、如何迭代的具体实践介绍得相对少。**

和内部运营、产品、研发小二聊得越多，我们越确信一件事：不能持续迭代的 Skill，说到底只是一次性的 demo。所以本文换个角度，从"架构设计、研发效能、运行优化、呈现方式"四个维度切入，以 `bybt-detail-assistant`（百亿补贴详情助手，50+ commit）的实际研发历程为例，介绍我们在迭代开发 **Skill** 过程中攒下的经验，给读者一些简单、能直接上手的 Skill 开发实践。

02 百亿补贴详情助手简介

`bybt-detail-assistant`（百亿补贴详情助手）是百亿补贴商品详情页的智能分析 Skill，由本地 **Skill** 文件和远程 **Agent** 两部分组成，两者之间通过 `xiaomi.py` 桥接脚本连接：



bybt-detail-assistant 的架构示意图

本地 Skill 的核心设计是"决策树路由 + 远程 **Agent API** 桥接"——本地负责信息收集、商品查询和结果呈现，而日志搜索、截屏分析这类依赖集团内部在线服务和知识库的深度分析，则交给远程 Agent。

层	运行环境	能力
本地 Skill 文件（QoderWork / 悟空）	用户本机	截图分析（识别商品名称/时间/价格/氛围）、商品查询（基于 curl 命令的商品中心搜索）
远程 Agent（工作流/ 智能助手 Agent）	远程 runtime	流量匹配（截图水印 → `traceId`）、完整埋点解读（百补超链业务模式、氛围命中/降级原因、券后价优惠组成等，整体仍在迭代）

03 业界 Skill 最佳实践梳理

3.1 Claude Skills：渐进式披露 + 控制调优

Anthropic 的官方最佳实践[1]和 Agent Skills 开放标准[3]建立了 Skill 设计的核心框架。社区对其做过 14 种设计模式的深度拆解[4]，其中以下几个关键理念对我们影响最大。

渐进式披露（Progressive Disclosure）[1][4]：

一般来说，Skill 的加载分三层：

- `L1 元信息（name + description）` → 每次会话启动时加载，决定 Skill 是否被激活；
- `L2 SKILL.md 正文` → 判断需要使用时才加载，是 Skill 的主指令集；
- `L3 附件资源（references/、scripts/）` → 执行过程中按需读取，不占用 L2 的 token 预算。

官方建议 `SKILL.md` 正文控制在 **500 行以内**，超过则拆分为独立文件[1]。其中，description 字段尤为关键，它不只是摘要，更是 Agent 在选择 Skill 时的重要信号。官方建议 description 要写得"有侵略性"（pushy），明确列出触发条件和排除条件，`bybt-detail-assistant` 的 description 覆盖了"截图/链接/ID/ `traceId` /找日志"等多种触发词。

控制调优（Control Tuning）[1][4]：

核心原则是**让指令自由度与任务鲁棒性相匹配**，Anthropic 官方文档中这一概念叫 *matching instruction freedom to task fragility*，文章[4]中称为 Control Tuning pattern，部分中文文章译为"自由度匹配"。

指令自由度	适用场景	bybt-detail-assistant 示例
低（精确命令，不可修改）	操作脆弱、容错率为零	强制以原格式输出详情访问报告
中（给出可替换参数的脚本）	有首选方法但允许变通	xiaomi.py 调用示例（只替换参数值）
高（只描述目标和约束）	多种方法都能到达	主动引导用户提问的话术

3.2 MiniMax Skills：标准化仓库 + 工业化生产

MiniMax 的开源仓库[2]是目前社区中成熟的 Skill 工业化样板（16+ 生产级 Skill，含 frontend-dev、fullstack-dev、android-native-dev 等），其工程化体系值得借鉴：

维度	做法	对我们的启发
结构校验	内置 validate_skills.py，PR 合入前自动验证 frontmatter 完整性、目录结构合规性	可参考建立 SKILL.md 的 CI 校验
多平台安装	同时提供 Claude Code / Cursor / Codex / OpenCode 四种安装方式	Skill 是平台无关的知识载体
命名规范	严格 kebab-case（连字符 "-" 分隔的小写单词），ASCII-only 文件名	保证跨平台兼容
组织方式	品牌前缀 + 能力（minimax-pdf）或 平台-类型（android-native-dev）	名称即自描述

另外，MiniMax 的 `CONTRIBUTING.md` 还要求"One PR, one purpose"：每个 PR 只做一件事（新增/修复/改进 Skill），这和我们 §4.2.3 的小步提交思路正好一致。

3.3 Skill 设计模式梳理

Skill 的设计模式是一个常被忽视但极有价值的维度。在对社区一批生产级 Skill 逐行分析的基础上，我们把它们归为两类：一类是决定 Skill 整体骨架的**工作流结构模式**，另一类是贯穿 `SKILL.md` 编写全过程的**高频实践模式**[4]。

先看 6 种核心的工作流结构模式：

模式	适用场景	代表 Skill	`bybt-detail-assistant` 的取舍
线性流程	部署、安装、迁移	vercel-deploy	路径 A/B 采用（步骤固定、命令明确）
决策树+按需加载	大型平台选型、问题诊断	cloudflare-deploy	✅ **整体采用。**由三条核心决策路径进行路由，除此之外，还为用户预留了自由提问的空间
循环迭代	TDD（测试驱动开发）、代码审查	test-driven-development	路径 C（日志搜索）的"搜索→判断→扩大窗口→再搜"循环
接力棒循环	跨 session 长期项目	stitch-loop	未采用
多阶段+检查点	复杂多周流程	discovery-process	未采用
思维框架	安全审计、代码审查、架构分析等需要结构化思考的分析任务	audit-context-building	未采用

`bybt-detail-assistant` 实际是**模式 2（决策树）为主干，路径内部混合模式 1（线性）和模式 3（循环迭代）**的复合结构。认清自己的 Skill 适合哪种模式，是结构设计的第一步，可以参考下图来选择。



Skill 工作流模式的决策树示意图

选择建议：如果你的 Skill 有"输入类型多样 → 处理逻辑互斥"的特征，优先考虑模式 2（决策树）；如果有"尝试 → 验证 → 重试"的特征，混入模式 3（循环迭代）。两者组合是诊断类 Skill 的常见范式。

在工作流骨架之外，还有几种贯穿 `SKILL.md` 编写的高频实践模式[4]，同样值得关注：

模式	一句话描述	我们的实践
已知陷阱（Known Gotchas）	在 `SKILL.md` 中维护一个"踩过的坑"列表，避免 Agent 重复犯错	`SKILL.md` 中的 FORBIDDEN 描述
正反示例（In-Skill Examples）	用 ✅ CORRECT / ❌ INCORRECT 对比锚定 Agent 行为	无损转述实践中的正反面示例
工具包模式（Utility Bundle）	把确定性逻辑封装到 scripts/ 中，只消耗输出 token	`search_detail_logs.py`、`xiaomi.py` 等工具脚本

有了上述业界方法论作为参考，接下来我们回到 `bybt-detail-assistant` 自身的迭代历程，逐一展开在架构设计、研发效能、运行优化和呈现方式四个维度中的具体做法。

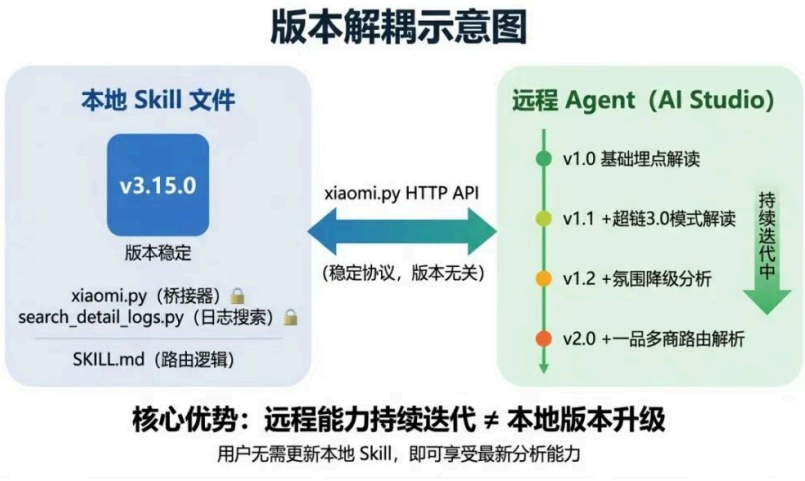
04 bybt-detail-assistant 技能研发最佳实践

4.1 架构设计

- 桥接器：解耦本地与远程的迭代

问题背景：`bybt-detail-assistant` 目前处在边试用边迭代的阶段，需要频繁升级（新增埋点字段解读、优化匹配算法）。如果把所有逻辑都放在本地 Skill，每次升级都要改动 `SKILL.md` 并重新发版，用户还得跟着频繁更新本地版本，很难保证所有用户体验一致。

我们的解法是：桥接器 `xiaomi.py` 本质是调用 Agent API（复用了原“百补详情小蜜 Agent”）客户端脚本。由于我们设计了稳定的协议，远程 Agent 升级时（如支持新埋点字段解读），Skill 作者无需修改本地 Skill。这就实现了"远程能力迭代"与"本地 Skill 版本"的解耦。



本地 Skill文件 与远程 Agent 版本解耦示意图

这套分离架构，说到底是在稳定性与迭代速度之间做的一次主动取舍，值得展开讲讲。

就缺点而言，最显而易见的代价就是跨系统桥接天然比单体更“脆”：本地与远程通过 `xiaomi.py` 约定的接口通信，多出一段网络链路和一层协议契约，任意一端的问题都可能让端到端链路失败；排查问题时也要在两个运行环境之间来回定位，归因成本更高。

但我们还是进行了拆分，根本原因很简单：这两部分的 **变化节奏** 和 **迭代成本** 完全不同。远程 Agent 扛的是分析能力，比如埋点解读、问题排查，几乎每周都在跟着新增的埋点和排查场景往上加。这类逻辑要是塞在本地，既慢，又容易版本不一致，还没有灰度的余地。放在远程 Agent 就不一样：版本管理、灰度发布、可观测、一键回滚都很成熟，天生适合安全又快速地上线。想试新逻辑，先灰度一小撮流量；出了问题，一键回滚，用户完全无感，也不用天天升级本地。本地 Skill 只保留信息收集、商品查询、结果呈现这层「薄客户端」，改动少，本就该是整条链路里最稳的一环。

就是基于上面的考虑，我们用 **架构边界** 把 **需要保持稳定** 和 **需要高频迭代** 两类诉求在物理上隔开，在远程分析能力还在高频迭代的当下，这道隔离带来的收益，远大于桥接本身的脆弱性。

- 决策树：按需加载而非全量展开

问题背景：我们的 `SKILL.md` 曾经把各个功能的描述交织在一起，在低端模型上经常让 Agent 顾此失彼、迷失方向。因此，我们把 `SKILL.md` 划分为三条逻辑互斥的路径（例如执行路径 A（输出详情访问报告）时，完全不需要知道路径 C（找详情访问日志）的细节），Agent 的执行效率明显提升。

我们的解法是：在共享全局约束的情况下，使用决策树拆分路径，每条路径自包含输出规则和主动引导逻辑。

```
全局约束（所有路径共用，~60行）
  ↓
决策树（4条规则，~20行）
  ├─ 路径A：traceId 分析（~30行）——线性模式
  ├─ 路径B：商品查询（~15行）——线性模式
  └─ 路径C：截图+日志（~250行）——循环迭代模式
```

这是模式 2（决策树+按需加载）在单文件内的实现——通过层级标题实现逻辑隔离。这种基于执行路径隔离的实践还有一个好处是**便于后续拆分**：目前百补详情助手的 `SKILL.md` 还尚未达到 500 行，未来继续膨胀时，可以方便地按照路径把文件物理拆分为 `path-a.md`、`path-b.md` 等，直接升级为完整的多文件渐进式披露。

- 注意力聚焦：减少 `SKILL.md` 中的次要信息

问题背景：最初尝试把远程 Agent 的所有分析规则（上百条埋点解读逻辑）写进 `SKILL.md`，结果本地 Agent 在执行简单的日志搜索时也要加载全量规则，模型 token 开销大，还容

易"走神"——偶尔会自作主张地去分析埋点，而不是老老实实搜日志；我们常常得打断 skill 执行、反复纠正，才能把 Agent 拉回正轨。

我们的解法是：对于埋点解读，本地 `SKILL.md` 只描述"什么时候调 `xiaomi.py`、参数怎么传、结果怎么输出"，不在 prompt 中塞入完整埋点分析规则。注意力开销从"全量规则库"缩减到"调用协议"，Agent 在执行当前步骤时注意力更集中。作为替代，我们用 Python 脚本固化埋点解读逻辑，经过实验，相比让 LLM 处理，脚本直出最多快 7 倍，且格式 100% 统一。这属于 Claude 所说的"低自由度"[1]——埋点解读报告有成熟的解析、展示规则，因此脚本可以直接固定"怎么输出"，把模型的注意力留给更重要的任务。

> 💡 Context Budget 意识[4]：当 `SKILL.md` 接近 500 行阈值时，每次新增内容都要问："删掉这句话会不会导致 Agent 出错？"如果不会，就不应该加。Agent 已经知道的常识不需要重复。

4.2 研发效能

- 直接用最强模型开发：让 AI 帮写 Skill

> 💡 这是所有实践中最简单、效率提升最明显的一条。

问题背景：一开始为了省 token，我们用中端模型来写 Skill，结果频繁出现理解偏差——路径 C 日志搜索的循环逻辑写得支离破碎，改了三四轮还不对，算下来消耗的总 token 远超一次性用强模型的开销。

随着实践的深入，我们越来越认同一个说法：在基于 Vibe Coding 进行 Skill 开发过程中，难点不在"怎么写文档"，而在"把需求讲清楚、把上下文喂足"。开发者的核心责任从从"手写文档"变成"描述意图 + 提供上下文 + 审查结果"之后，开发过程很最关键的一个变量，就是模型本身的能力。

我们的解法是：既然创建和优化 Skill 本身就是高复杂度任务，开发阶段直接用最强模型一步到位，把官方 Skills 最佳实践[1] + 真实案例数据（包括报错信息） + 现有 `SKILL.md` 一并放入 Agent 上下文，要求 ta 解决当前问题/实现预期功能（**当然如果可以提供正确的解决思路会更好:D**）。然后让 ta 反思、总结自己解决问题的过程，自己输出 Skill。最终到了验证时，再测试弱模型的兼容性，保证 Skill 在各种环境下都能按预期稳定执行。

- 本地远程共享视角：按需暴露上下文

问题背景：本地 Skill 调用远程 Agent 返回结果后，经常出现格式解析失败——原因是本地开发时不了解远程 workflow 的返回结构变化。本地桥接器期望的 JSON schema 和远程实际返回的结构之间产生偏差，就像前后端接口协议不同步一样痛苦。

我们的解法是：借鉴 **Monorepo** 的核心实践——将跨边界代码纳入同一可见范围。



Skill 开发迭代的策略示意图

Google 的经典论文 "*Why Google Stores Billions of Lines of Code in a Single Repository*"[5] 总结了 Monorepo 的核心优势：统一版本视图、跨项目原子性修改、简化依赖管理。在我们的场景中，虽然本地 Skill 和远程 Agent 物理上分属两个系统，但我们将远程 Agent 的代码（FC-style 节点 + LLM 节点 prompt）纳入 Agent 的同一工作区，让 ta 拥有完整的上下文：

Monorepo 优势	在 Skill 研发中的体现
统一版本视图	本地脚本和远程节点代码在同一 workspace，Agent 能看到完整上下文
跨项目原子修改	修改远程返回格式时，Agent 同步更新本地解析逻辑，不会遗漏
简化依赖管理	编写 `xiaomi.py` 时直接参考远程节点的输出结构，无需跨平台查看

但在实践当中，我们也同时注意到 Monorepo 的经典取舍在 AI 场景下被放大了：**全局视角 vs 注意力分散**。Google 的论文[5]就指出 Monorepo 的挑战之一是"工具必须能处理大规模代码"，而在 AI Agent 场景下，这个问题变成了"上下文窗口必须能处理大规模信息"。我们遇到的问题具体表现为：

1. **上下文污染导致注意力分散**：例如，当我们只想修改本地的商品搜索策略时，Agent 之前读取的无关内容（例如：远程 workflow 的埋点解析规则）被无效地填入上下文窗口，白白消耗模型的注意力。
2. **越界修改**：Agent 看到远程代码后，有时会"好心"地顺便“优化”远程节点逻辑，导致本地和远程的改动纠缠在一起，难以独立回滚。

我们的平衡策略是**按任务需要动态切换工作区范围，而不是把所有代码常驻在同一上下文中**：

- **常规迭代**（只改本地 or 只改远程）：另起独立的 Coding Agent 会话，只加载当前侧的代码，避免上下文污染。
- **协议设计 / 边界调整**（本地+远程同时改）：先让 Coding Agent 反思两侧的新增改动，然后切换为共享视角进行跨边界的原子性修改。

这本质上呼应了 Claude Code 官方最佳实践[6]所倡导的聚焦当前任务上下文的理念——平时缩小范围，需要时才扩大视野。

- 小步提交：让每次改动可追溯、可回滚

问题背景：早期我们经常一口气改完多个路径的逻辑，结果一旦出了问题，很难定位是哪处改动导致的，这导致回滚成本极高。

我们的解法是：使用 Git 管理 `skills/` 目录，每次迭代只改一个维度（比如只调整时间窗口策略，或只修一个术语），改完验证通过后，立即 提交 commit。diff 越小，AI 辅助审查就越精准—— ta 能明确告诉你"这次修改是否引入了信息遗漏"，这与 §3.2 提到的 MiniMax "One PR, one purpose"[2] 理念一致。

- 让 Agent 看到"运行时"：提供真实数据和完整报错

问题背景：我们踩过两个坑：1、开发时 Agent 使用普通商品数据调试日志搜索功能一切正常，实际使用时才发现百补热门商品（例如：iPhone 17 系列）1 分钟内可能有上千条访问记录，默认 5 分钟的搜索窗口直接返回爆炸数据量。2、`search_detail_logs.py` 遇到字段缺失、编码异常、JSON 嵌套层级变化时报错频出，只把报错的 `ErrorMessage` 丢给 Agent，ta 并不理解报错进程的真正上下文，几轮修改后问题仍在。经过分析，我们意识到这 2 个坑实际上对应同一个原因：我们没给 ta 系统运行时的充足上下文。

我们的解法是：把运行时现场原封不动交给 Agent。

强调 Agent 端到端验证时，一律用包含热点的真实商品和日志，不再用虚构数据——只有让 Skill 跑在真实流量分布下，那些真实问题（包括热门商品、罕见氛围组合、跨单元链路等长尾场景）才会浮出水面。对于真实场景下暴露的极端 case，我们直接指导 Agent 进行评估，如果有价值，则立刻在 `SKILL.md` 里将其固化为一个 `gotcha`，把这份"运行时才发现的事"变成下次执行一开始就带着的经验。

脚本异常时也一样：不要只丢一行 `ErrorMessage`，而是把错误堆栈 + 源码 + 触发这次异常的那一份具体输入数据一次性打包给 Agent。如此一来，源码提供业务逻辑上下文、堆栈提供报错关键点、输入数据提供复现现场——三样凑齐才是完整的运行时快照。有了这份快照，Agent

会自己在"改一版 → 用同一份输入回放 → 再改"的 `Loop` 里收敛，比人工逐行看日志 debug 快得多。

4.3 运行优化

- 强制"原文直出"：对抗 LLM 的"总结冲动"

问题背景：如 `4.1.3` 小节所述，为了节省模型 Attention、降低返回时延，我们在远程 Agent 中用脚本生成 Markdown 格式的标准化分析报告，但本地模型拿到工具返回后，经常对报告进行二次加工——添加标题、改写措辞、截断表格，甚至添油加醋地补充不存在的分析结论。然而，在日志分析场景中，几乎任何"总结"都是信息损失。

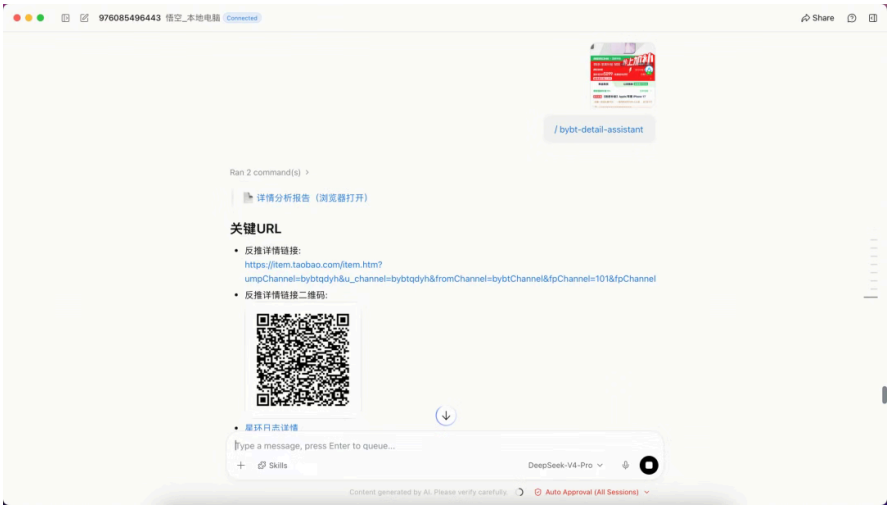


“原文直出”对比示意图

我们的解法是：在 `SKILL.md` 中用极强语气定义禁止列表：

```
FORBIDDEN actions:
- ❌ 不要对 stdout 内容进行总结或改写
- ❌ 不要重新排版或调整 Markdown 格式
- ❌ 不要在 stdout 前后添加标题、解释或上下文
✅ CORRECT: 直接输出 stdout，一字不改
❌ INCORRECT: "根据分析结果，该商品..."
```

我们限制 SKILL 输出后的实际效果如下：截图输入 → 远程分析 → 结构化报告原样直出



路径A原文直出截图

- `sessionId` 会话复用：维持远程 Agent 的上下文连贯

问题背景：如前所述，百亿补贴详情助手分为本地文件（负责处理用户输入、商品搜索等）和远程 Agent（日志搜索、埋点解析），早期每次调用远程 Agent 时，我们发现远程 Agent 总是"失忆"——用户先发截图让远程分析，再追问"刚才截图中的详情页面为什么无法加购？"，解决这个问题的日志信息存储在远程 Agent，然而远程 Agent 却完全不记得之前分析过什么日志，用户只能重新发一遍截图。

我们的解法是：在 `SKILL.md` 中规定——每个会话的第一次调用 `xiaomi.py` 前生成一个随机 `sessionId`（格式 `session-<8位字母数字>`），此后该会话内所有后续调用**复用同一个 ID**。远程 Agent 通过 `sessionId` 识别同一对话上下文，实现跨调用的记忆保持。这是一个极小的设计（`SKILL.md` 中只增加了 ~10 行），但对用户体验的提升非常显著。

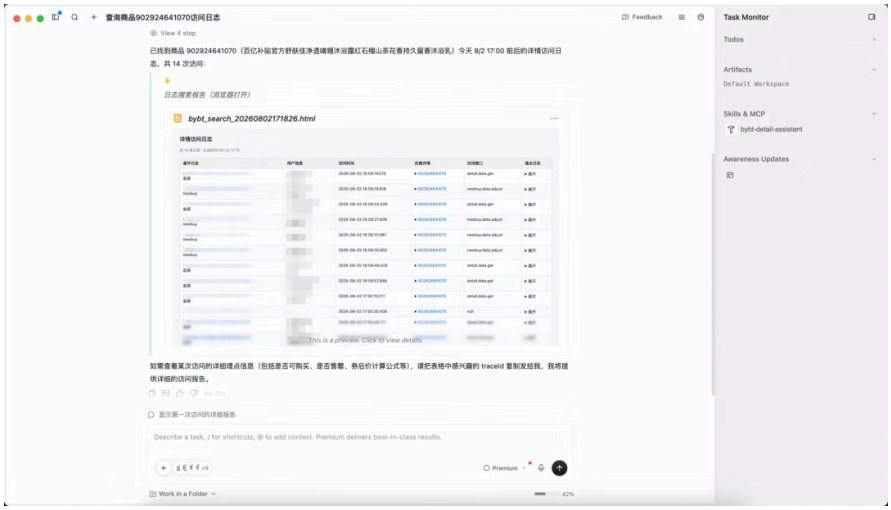
- 优先给出 HTML 报告：提高信息密度

问题背景：一次日志搜索（路径 C）可能返回几十条记录，详情访问报告（路径 A）也常常占满两屏。如果全部展开在对话里，用户要翻好几屏才能找到关心的那条，既容易信息过载，也得等很久才能看到自己真正关注的内容。

我们的解法是：对于报告类型的长内容回复，本地 Skill 文件（`SKILL.md` 和脚本）直接处理为 html 文件，优先输出本地 html 报告路径，然后让 Agent 自己挖掘其中的关键信息进行展示，例如：

```
> 👆 [日志搜索报告（浏览器打开）](file:///tmp/bybt_search_xxx.html)
后续：  Markdown 表格（折叠+链接）
```

这样用户点击之后，可以直接在浏览器里打开报告等排版良好的页面，然后借助浏览器对 HTML 的完整支持做搜索等操作。实际效果如下：



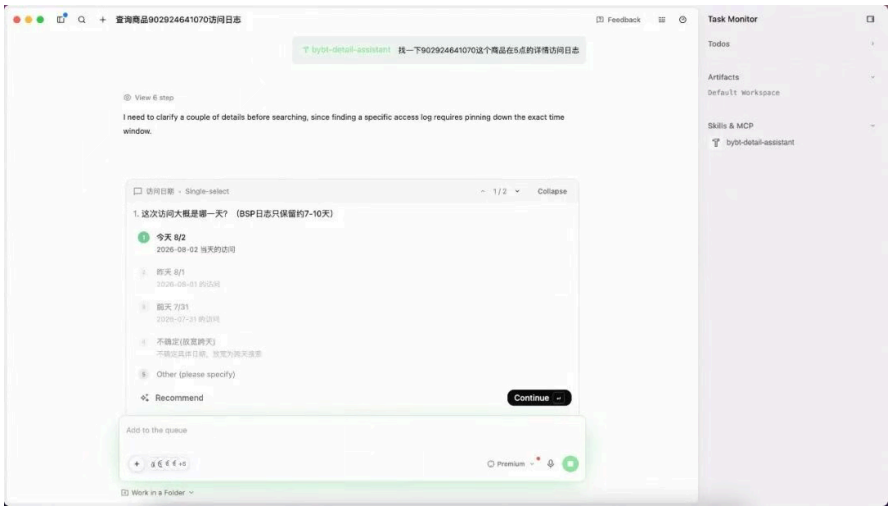
HTML 文件路径 + Markdown 回复

注意：这里一定要让本地 Agent 来执行 html 文件转换，否则如果放到远程 Agent 产出报告链接，本地 Agent 很可能会丢失报告本身的内容信息，把这种信息密度高、价值大的上下文忽略掉，影响后续轮次的交互。

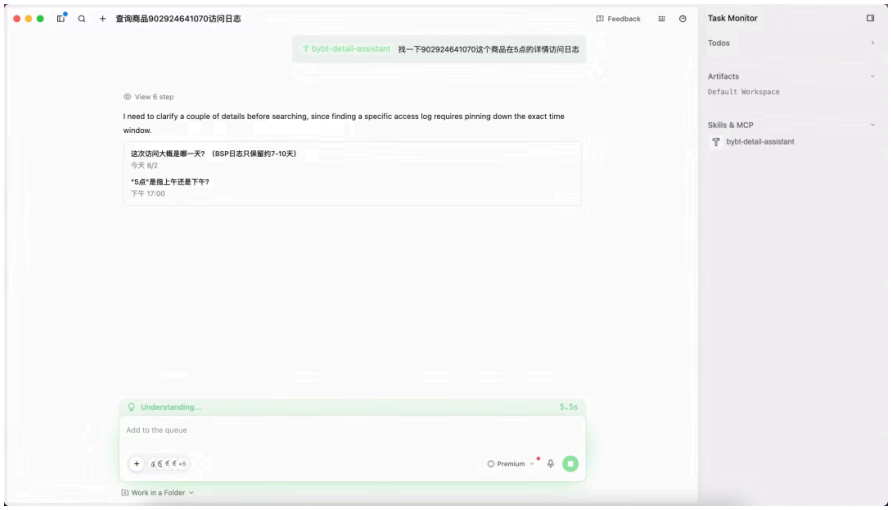
- 主动引导：Agent 该问的先问清，用户没问到的主动递

问题背景：主动引导要处理一头一尾两种场景。一种在开头：用户丢来一张截图问"帮我看看这次访问"，却没说是哪天、是预发还是线上——定位日志需要的关键前提缺了一块。Agent 要是闷头按默认值猜，很可能搜出一堆无关记录，用户白等一场。另一种在结尾：日志里其实藏着 `regionLimit=true`（区域限售）、`buyNow=false`（无法立即购买）、`atm=false`（未命中百补氛围）这些信号，可用户只问了"这个商品正不正常"，你不主动点出来，ta 根本不知道真正的问题在哪。

我们的解法是：执行前，如果发现关键信息缺失导致无法定位到请求，那么先问清再动手。问也讲方式：使用 `AskUserQuestion` 将多个疑点合并成一轮，每个都配好选项和默认值，用户点一下或回一个词就行；如果用户实在拿不准选了"不确定"，就自动放宽搜索范围（跨天、上午下午全覆盖），并如实告诉 ta 这次搜得比较宽。反过来，能从截图或上下文推断出来的（像 24 小时制的"20:11"一看就是下午）绝不多此一问——澄清是为了让用户少等一个错误结果，而非机械式地填表。



执行日志搜索前主动澄清时间范围（1）



执行日志搜索前主动澄清时间范围（2）

执行后，Agent 再扫一遍分析结果，命中异常信号就在报告末尾追一句引导式提问，比如"检测到该商品区域限售（`regionLimit=true`），要不要看看具体限售区域？"。把所有分析一股脑倒给用户，反而会淹没最要紧的那 3 条；所以我们只递出问题，深挖不深挖，用户自己定。这也正好落在 §3.1 控制调优表的"高自由度"一档——只给 Agent 定目标点出异常，不写死话术，具体怎么问让 ta 临场发挥。

05 结语

回顾百补详情助手迭代历程，我们有 2 点反思最想分享：

第一，Skill 的质量上限由"上下文工程"能力决定，而非写作技巧。在对的时机（Agent 要做判断的那一刻），把对的信息（真实案例、错误堆栈、API 返回结构），以对的粒度（不多不少，刚好够做决策）递到模型面前——对于 Skill 迭代而言，这才是真正的核心竞争力，而 `SKILL.md` 只是这种能力的载体之一。

第二，让 Agent 自己写 Skill、自己改 Skill，是一种"元能力"。当你把写 Skill、改 Skill 这件事本身也交给 Agent，人类开发者就从"编写者"变成了"审查者 + 上下文提供者"。这带来的不只是效率，还是一种全新的人机协作方式。

Code is Cheap，Show Me Your Skills! ——让我们一起构建可以带来实际提效的 Skills!

06 参考文献

[1] Anthropic. Skill authoring best practices. https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices

[2] MiniMax-AI. MiniMax Skills - Official Open-Source Skills for AI Coding Agents. https://github.com/MiniMax-AI/skills

[3] Agent Skills. Open Standard for Agent Skills. https://agentskills.io

[4] Generative Programmer. Skill Authoring Patterns from Anthropic's Best Practices. https://generativeprogrammer.com/p/skill-authoring-patterns-from-anthropics

[5] Rachel Potvin & Josh Levenberg. Why Google Stores Billions of Lines of Code in a Single Repository. Communications of the ACM, 2016. https://cacm.acm.org/research/why-google-stores-billions-of-lines-of-code-in-a-single-repository/

[6] Anthropic. Claude Code: Best practices for agentic coding. https://www.anthropic.com/engineering/claude-code-best-practices

07 团队介绍

本文作者曲士，来自淘天集团-天猫技术团队。百亿补贴与聚划算是淘天面向价格敏感用户与品牌商家的核心营销阵地，也是我们直面市场竞争最前沿的"主战场"；我们不断"刷新"技术能力，以半托管、竞价与补贴系统的建设，支撑百亿补贴成为淘天近三年增长最快的业务。如今我们正用 AI 重构从招商审核到补贴发放的运营全流程，推动"人工配置"向"AI 自治"演进，并把研发链路重塑为需求分析、代码生成到测试上线的端到端 AI 原生范式。



大淘宝技术

大淘宝技术官方账号

966篇原创内容

公众号

阅读 8596

留言

写留言

还没有留言，来写首评